Architecture des Ordinateurs : Représentation des Entiers et Arithmétique en Complément à Deux

Thème principal : Représentation des entiers signés et arithmétique binaire

Synthèse exécutive : Ce cours aborde la manière dont les ordinateurs codent et manipulent physiquement les entiers en mémoire. En l'absence de composants électroniques capables de stocker directement un signe '+' ou '-', l'architecture informatique utilise la convention du complément à deux sur un nombre fixe de bits. Ce système unifie l'addition et la soustraction au niveau de l'Unité Arithmétique et Logique (ALU) tout en éliminant le problème du double zéro. Nous détaillons la méthode de conversion pas à pas, les opérations arithmétiques bas niveau, ainsi que la détection des erreurs de dépassement de capacité (overflow), piège classique lors des contrôles.

Problématique matérielle et justification du Complément à Deux

Au niveau le plus bas de l'architecture informatique, l'information est stockée dans des transistors et des bascules bistables ne reconnaissant que deux états logiques (0 pour 0V et 1 pour Vcc). Un ordinateur ne possède aucun moyen d'inscrire directement un glyphe comme '-' ou '+'. Pour représenter des entiers relatifs, les concepteurs ont d'abord envisagé le codage 'Signe + Magnitude' (où le bit le plus à gauche indique le signe et les autres la valeur absolue). Cependant, ce codage génère deux représentations pour zéro (+0 et -0), ce qui gaspille une combinaison binaire et complexifie les tests d'égalité. De plus, il impose d'ajouter un circuit de soustraction distinct dans l'ALU. Le complément à deux résout totalement ces problèmes : zéro possède une représentation unique (0000...0) et l'opération de soustraction A - B devient une simple addition A + (-B), réutilisant le circuit d'addition binaire existant.

- Absence de signe physique dans les transistors et bascules bistables
- Limites du codage 'Signe et Magnitude' (double zéro et circuits séparés)
- Unification de l'ALU : la soustraction ramenée à une addition

Cas concret enseignant : *Le professeur utilise la métaphore du compteur kilométrique d'une voiture ancienne à 4 chiffres : si le compteur est bloqué à 0000 km et qu'on effectue un kilomètre en marche arrière, le compteur affiche 9999. En binaire sur 8 bits, retrancher 1 à 00000000 donne 11111111_2, qui définit la représentation de -1 en complément à deux.*

Algorithme de Conversion et Plage de Valeurs sur N bits

Pour convertir un nombre négatif -X en complément à deux sur un mot mémoire de N bits, il faut suivre une méthode stricte en trois étapes. Premièrement, on écrit la valeur absolue X en binaire pur sur N bits complets (en rajoutant les 0 nécessaires à gauche). Deuxièmement, on applique le complément à un en inversant chaque bit (les 0 deviennent 1, les 1 deviennent 0). Troisièmement, on ajoute 1 au résultat binaire. La plage des nombres représentables sur N bits n'est pas symétrique : sur 8 bits par exemple, elle s'étend de -128 à +127. Cette dissymétrie provient du fait que le zéro est inclus dans le domaine positif (MSB = 0).

- Méthode des 3 étapes : Valeur absolue, Inversion (Complément à 1), +1
- Rôle du bit MSB (Most Significant Bit) pour le signe
- Dissymétrie de la plage de valeurs : de -2^(N-1) à +(2^(N-1) - 1)

Cas concret enseignant : *Avertissement crucial du professeur pour les partiels : il faut impérativement fixer la taille de N bits (ex: 8 bits) AVANT d'effectuer l'inversion des bits. Inverser '1100' sans mettre au préalable les zéros de gauche '00001100' produit un résultat complètement faux.*

Arithmétique, Retenue (Carry) et Détection du Dépassement de Capacité (Overflow)

En travaillant sur des registres de taille fixe (ex: 8, 16, 32 ou 64 bits), les calculs arithmétiques peuvent dépasser la capacité maximale du registre. Il est vital de distinguer le Carry Out (retenue sortant du bit le plus à gauche) du drapeau d'Overflow (OV). Lors d'une addition en complément à deux sur N bits, le Carry Out est simplement ignoré car le calcul s'effectue modulo 2^N. L'Overflow, en revanche, signale que le résultat dépasse la plage représentable (-2^(N-1) à +2^(N-1)-1). La règle formelle de détection d'un Overflow en examen est : un Overflow survient si et seulement si deux nombres de même signe additionnés produisent un résultat de signe inverse. L'addition de deux nombres de signes différents ne peut jamais générer d'Overflow.

- Distinction entre Carry Out (retenue sortante) et Overflow
- Règle de détection de l'Overflow par les signes des opérandes
- Impossibilité d'overflow lors de l'addition de signes opposés

Cas concret enseignant : *L'anecdote historique du crash d'Ariane 5 (vol 501 en 1996) : un nombre flottant sur 64 bits a été converti en entier signé 16 bits sans protection contre le dépassement de capacité. L'Overflow non géré a provoqué la défaillance du système de guidage et la destruction de la fusée.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi la retenue tout à gauche (Carry sortant) est-elle jetée lors de l'addition en complément à deux ?
Réponse du professeur : Parce que le processeur travaille sur une taille de registre fixe (modulo 2^N). La retenue sort du cadre matériel du registre, mais mathématiquement, l'arithmétique modulo 2^N gère les nombres négatifs et positifs correctement sans cette retenue.

Question étudiant : Comment l'ordinateur fait-il la différence entre le nombre binaire 11111111 qui vaut 255 en non-signé et -1 en complément à deux ?
Réponse du professeur : L'ordinateur ne fait pas de différence physique sur les bits eux-mêmes : c'est le contexte de l'instruction machine (ou le choix du programmeur) qui indique si l'ALU doit interpréter le registre comme un entier signé ou non-signé.

■ Exercice & Démonstration d'Application

Énoncé : Sur une architecture 8 bits, effectuer la soustraction signée : 35 - 75. Détailler l'écriture binaire des opérandes, effectuer l'addition binaire correspondante, donner le résultat décimal et indiquer s'il y a eu un Overflow.

Résolution pas à pas :
1. Conversion de +35 sur 8 bits : 35 = 32 + 2 + 1 = 00100011_2. 2. Conversion de -75 en complément à deux sur 8 bits : - Valeur absolue +75 sur 8 bits : 64 + 8 + 2 + 1 = 01001011_2. - Inversion des bits (Complément à 1) : 10110100_2. - Ajout de 1 (Complément à 2) : 10110100 + 1 = 10110101_2 (-75 décimal). 3. Pose de l'addition (+35) + (-75) : 00100011 (+35) + 10110101 (-75) ---------- 11011000_2 4. Interprétation du résultat : Le bit MSB est à 1, le résultat est donc négatif. Pour retrouver sa valeur absolue, on applique le complément à deux sur 11011000_2 : - Inversion : 00100111_2 - Ajout de 1 : 00101000_2 = 32 + 8 = 40 décimal. Le résultat binaire 11011000_2 représente donc -40 en décimal. 5. Analyse de l'Overflow : On additionne un nombre positif (+35, MSB=0) et un nombre négatif (-75, MSB=1). L'addition de deux nombres de signes opposés ne peut jamais produire d'Overflow.

Résultat : **Résultat binaire : 11011000_2 (-40 en décimal). Drapeau d'Overflow = 0. Le calcul 35 - 75 = -40 est exact et valide.**

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la plage des entiers signés représentables sur un mot de 16 bits en complément à deux ?

R : De -32768 à +32767 (soit -2^15 à 2^15 - 1).

Q : Comment détecte-t-on de façon certaine un Overflow lors de l'addition de deux entiers signés ?

R : Lorsque deux opérandes de même signe produisent un résultat dont le signe (MSB) est opposé.

Q : Quel est le résultat de l'inversion et de l'ajout de 1 (complément à deux) appliqué au nombre zéro (00000000_2) sur 8 bits ?

R : On obtient à nouveau 00000000_2 (avec un carry out de 1 ignoré), ce qui prouve l'unicité du zéro.